



INSTITUTO TECNOLÓGICO SUPERIOR DE LERDO.



- **Alumno:**

Rodriguez Rosales Diego Guillermo.

- **Num. Control:**

#222310306.

- **Docente:**

Jesus Salas Marin.

- **Materia:**

Desarrollo de Aplicaciones para Dispositivos Móviles.

- **Clave:**

4Y8A.

- **Actividad:**

Pra 2.4.8 Implementando Herencia en Dart.

20-Abril-2026.



Instrucciones.

1. Modifica el código base:

```
1 // Clase base Vehiculo
2 class Vehiculo {
3     String marca;
4     String modelo;
5     int anio;
6
7     Vehiculo(this.marca, this.modelo, this.anio);
8     void mostrarDetalles() {
9         print("Vehículo: $marca $modelo ($anio)");
10    }
11 }
12
13 // Clase Auto (hereda de Vehiculo)
14 class Auto extends Vehiculo {
15     int numeroPuertas;
16     Auto(String marca, String modelo, int anio, this.numeroPuertas)
17         : super(marca, modelo, anio);
18
19     @override
20     void mostrarDetalles() {
21         super.mostrarDetalles();
22         print("Número de puertas: $numeroPuertas");
23     }
24 }
25
26 // Clase Camion (hereda de Vehiculo)
27 class Camion extends Vehiculo {
28     double capacidadCarga; // en toneladas
29
30     Camion(String marca, String modelo, int anio, this.capacidadCarga)
31         : super(marca, modelo, anio);
32
33     @override
34     void mostrarDetalles() {
35         super.mostrarDetalles();
36         print("Capacidad de carga: $capacidadCarga toneladas");
37     }
38 }
39
40 // Función principal
41 void main() {
42     Auto miAuto = Auto("Toyota", "Corolla", 2020, 4);
43     Camion miCamion = Camion("Volvo", "FH16", 2018, 20.5);
44
45     print("Detalles del Auto:");
46     miAuto.mostrarDetalles();
47
48     print("\nDetalles del Camión:");
49     miCamion.mostrarDetalles();
50 }
```



- Haciendo las propiedades privadas.
- Agrega los setters y getters.
 - Luego, ejecuta el código y observa cómo se comportan los objetos heredados.

```
1 // Clase base Vehículo
2 class Vehículo {
3     String _marca;
4     String _modelo;
5     int _anio;
6
7     Vehículo(this._marca, this._modelo, this._anio);
8
9     // Getters
10    String get marca => _marca;
11    String get modelo => _modelo;
12    int get anio => _anio;
13
14    // Setters
15    set marca(String marca) => _marca = marca;
16    set modelo(String modelo) => _modelo = modelo;
17    set anio(int anio) => _anio = anio;
18
19    void mostrarDetalles() {
20        print("Vehículo: $_marca $_modelo ($_anio)");
21    }
22 }
23
24 // Clase Auto
25 class Auto extends Vehículo {
26     int _numeroPuertas;
27
28     Auto(String marca, String modelo, int anio, this._numeroPuertas)
29         : super(marca, modelo, anio);
30
31     int get numeroPuertas => _numeroPuertas;
32     set numeroPuertas(int numeroPuertas) =>
33         _numeroPuertas = numeroPuertas;
34
35     @override
36     void mostrarDetalles() {
37         super.mostrarDetalles();
38         print("Número de puertas: $_numeroPuertas");
39     }
40 }
41
42 // Clase Camión
43 class Camión extends Vehículo {
44     double _capacidadCarga;
45
46     Camión(String marca, String modelo, int anio, this._capacidadCarga)
47         : super(marca, modelo, anio);
48
49     double get capacidadCarga => _capacidadCarga;
50     set capacidadCarga(double capacidadCarga) =>
51         _capacidadCarga = capacidadCarga;
52
53     @override
54     void mostrarDetalles() {
55         super.mostrarDetalles();
56         print("Capacidad de carga: $_capacidadCarga toneladas");
57     }
58 }
59
60 // Función principal
61 void main() {
62     Auto miAuto = Auto("Toyota", "Corolla", 2020, 4);
63     Camión miCamión = Camión("Volvo", "FH16", 2018, 20.5);
64
65     print("Detalles del Auto:");
66     miAuto.mostrarDetalles();
67
68     print("\nDetalles del Camión:");
69     miCamión.mostrarDetalles();
70 }
```



- Luego, ejecuta el código y observa cómo se comportan los objetos heredados.

```
PS C:\Users\dgrr2\analisis_datos\bin> dart run Pra2.4.8.dart
Detalles del Auto:
Vehículo: Toyota Corolla (2020)
Número de puertas: 4

Detalles del Camión:
Vehículo: Volvo FH16 (2018)
Capacidad de carga: 20.5 toneladas
```

Preguntas

1. ¿Qué ventajas tiene la herencia en la programación?

La herencia permite reutilizar código, evitar duplicación, organizar mejor las clases y facilitar el mantenimiento del programa. También permite aplicar el polimorfismo, haciendo el sistema más flexible y escalable.

2. ¿Cómo se usa la palabra clave super en Dart?

La palabra clave super se usa para acceder a los atributos o métodos de la clase padre.

3. ¿Qué sucede si no sobrescribimos mostrarDetalles() en las clases hijas?

Si no se sobrescribe el método, las clases hijas usarán el método de la clase padre y solo mostrarán la información básica del vehículo, sin incluir las propiedades adicionales como número de puertas o capacidad de carga.



Desafío.

- Agregar una clase Motocicleta que herede de Vehículo, con una propiedad tipo ("deportiva", "crucero", etc.), luego prueba la clase Motocicleta.

```
1 // Clase Motocicleta (Desafío)
2 class Motocicleta extends Vehiculo {
3     String _tipo;
4
5     Motocicleta(String marca, String modelo, int anio, this._tipo)
6         : super(marca, modelo, anio);
7
8     String get tipo => _tipo;
9     set tipo(String tipo) => _tipo = tipo;
10
11     @override
12     void mostrarDetalles() {
13         super.mostrarDetalles();
14         print("Tipo: $_tipo");
15     }
16 }
17
18 Motocicleta miMoto =
19     Motocicleta("Yamaha", "R6", 2022, "Deportiva");
20
21 print("\nDetalles de la Motocicleta:");
22 miMoto.mostrarDetalles();
```

```
PS C:\Users\dgrr2\analisis_datos\bin> dart run Pra2.4.8.dart
Detalles del Auto:
Vehículo: Toyota Corolla (2020)
Número de puertas: 4

Detalles del Camión:
Vehículo: Volvo FH16 (2018)
Capacidad de carga: 20.5 toneladas

Detalles de la Motocicleta:
Vehículo: Yamaha R6 (2022)
Tipo: Deportiva
```



Conclusión.

En esta práctica aprendimos a implementar la herencia en Dart mediante la creación de una clase base llamada Vehículo y clases hijas como Auto, Camión y Motocicleta. Comprendimos cómo reutilizar código utilizando la palabra clave `extends` y cómo sobrescribir métodos mediante `@override` para agregar comportamientos específicos en cada clase hija.

También reforzamos el concepto de encapsulamiento al hacer las propiedades privadas y utilizar getters y setters para controlar el acceso a los atributos. Esto mejora la seguridad, organización y mantenimiento del código.